Assignment 4

1. What is a dictionary in Python? List out all properties of Dictionary.

```
A dictionary in Python is a built-in data type used to store data
in the form of key-value pairs.
Each key is associated with a value, and we use the key to access
its corresponding value.
```

Properties of Dictionary in Python

1. Mutable: We can add, update, and delete elements after dictionary creation.
2. Ordered: Dictionaries preserve the insertion order of key-value pairs (Python 3.7 onwards).
3. Key-Value Pair: Dictionary stores data in the form of keys and values.
4. Unique Keys: Duplicate keys are not allowed. If duplicate keys are provided, the latest value overwrites the previous value.
5. Keys Must Be Immutable (Hashable): Keys can be strings, integers, floats, or tuples containing hashable elements. Lists, sets, and dictionaries cannot be used as keys.
6. Values Can Be of Any Data Type: Values can be integers, strings, lists, tuples, sets, dictionaries, etc.
7. Heterogeneous Data: A dictionary can contain different types of keys and values.
8. Dynamic in Size: We can increase or decrease the size of a dictionary at runtime.
9. Nested Dictionary: A dictionary can contain another dictionary as its value.
10. Accessed Using Keys: Dictionary elements are accessed using their keys rather than indexes.
11. Keys Are Case-Sensitive: "Name" and "name" are treated as two different keys.
12. No Slicing: Dictionaries do not support indexing and slicing like lists and tuples.
13. Duplicate Values Allowed: Multiple keys can have the same value.

2. Explain the concept of dictionary comprehension in Python.

```
Dictionary comprehension is a concise way of creating a dictionary
in a single line of code using an expression and an iterable.
It allows us to create dictionaries without writing lengthy for
loops.
```

```
d = {i: i**2 for i in range(1, 6)}
```

3. Explain the difference between the dict.get(key) method and accessing dict[key] directly

```
In Python, both dict.get(key) and dict[key] are used to retrieve
values from a dictionary using their keys.
```

However, the main difference is how they behave when the specified key does not exist.

1. Using dict[key] (Direct Access)

- Retrieves the value associated with the specified key.
- If the key exists, it returns the corresponding value.
- If the key does not exist, it raises a KeyError exception.

2. Using dict.get(key) Method

- Retrieves the value associated with the specified key.
- If the key exists, it returns the corresponding value.
- If the key does not exist, it returns None by default instead of raising an error.
- We can also specify a default value that will be returned if the key is missing.

4. How can you merge two dictionaries in Python?

In Python, merging dictionaries means combining the key-value pairs of two or more dictionaries into a single dictionary.

We can merge dictionaries using the following methods:

1. Using update() method

- Adds all key-value pairs of one dictionary into another dictionary.
- Modifies the original dictionary.
- Does not create a new dictionary.
- Returns None.
- If duplicate keys exist, values from the second dictionary overwrite the values of the first dictionary.

2. Using Dictionary Unpacking (**)

- Uses the unpacking operator (**) to combine dictionaries.
- Creates a new dictionary.
- Does not modify the original dictionaries.
- If duplicate keys exist, the rightmost dictionary's value overwrites the previous value.

3. Using Union Operator (|)

- Introduced in Python 3.9.
- Combines two dictionaries into a new dictionary.
- Does not modify the original dictionaries.
- If duplicate keys exist, the right-hand dictionary's value takes priority.

4. Using Union Assignment Operator (|=)

- Introduced in Python 3.9.
- Merges one dictionary into another.
- Modifies the original dictionary.
- Does not create a separate dictionary.

In [2]:
```
### 5. How can you update dictionary values in Python?
"""In Python, we can update dictionary values using different methods. A dictionary

The following methods can be used to update dictionary values:

1. Using Assignment Operator (=)
- We can update a value by accessing its key and assigning a new value.
- If the key already exists, its value gets updated.
- If the key does not exist, a new key-value pair is added.
2. Using update() Method
- The update() method is used to update one or more key-value pairs.
- It can accept another dictionary or an iterable of key-value pairs.
- If a key already exists, its value gets overwritten.
- If a key does not exist, a new key-value pair is added.
- It modifies the original dictionary and returns None.
3. Using setdefault() Method
- The setdefault() method returns the value of the specified key if it already exis
- If the key does not exist, it inserts the key with the specified default value.
- It does not overwrite an existing value.
4. Using Dictionary Unpacking (**)
- We can update dictionary values by creating a new dictionary using the unpacking
- The original dictionary remains unchanged.
- Values from the rightmost dictionary overwrite duplicate keys.
Examples
1. Using Assignment Operator"""
d = {"a": 10, "b": 20}
d["a"] = 100
print(d)

#2. Using update() Method
d = {"a": 10, "b": 20}
d.update({"a": 100, "c": 30})
print(d)

#3. Using setdefault() Method
d = {"a": 10, "b": 20}
d.setdefault("c", 30)
print(d)

#4. Using Dictionary Unpacking
d = {"a": 10, "b": 20}
d1 = {**d, "a": 100}
print(d1)
```

```
{'a': 100, 'b': 20}
{'a': 100, 'b': 20, 'c': 30}
{'a': 10, 'b': 20, 'c': 30}
{'a': 100, 'b': 20}
```

6. How can you iterate over a dictionary to access keys, values, and key-value pairs?

In Python, we can iterate over a dictionary using a for loop to access its keys, values, and key-value pairs.

Python provides the following methods for dictionary iteration:

1. Using keys() Method

- The keys() method returns all the keys present in a dictionary.
- We can iterate over the keys using a for loop.

2. Using values() Method

- The values() method returns all the values present in a dictionary.
- We can iterate over the values using a for loop.

3. Using items() Method

- The items() method returns all key-value pairs in the form of tuples.
- We can use two variables in a for loop to access keys and values separately.

4. Directly Iterating Over a Dictionary

- By default, iterating over a dictionary directly returns its keys.
- It is equivalent to iterating using the keys() method.

7. What are the different ways to delete items from a dictionary in Python?

In Python, we can delete items from a dictionary using different methods and statements. Since dictionaries are mutable, we can remove key-value pairs after dictionary creation.

The following methods are used to delete items from a dictionary:

1. Using del Keyword

- The del keyword is used to delete a specific key-value pair from a dictionary.
- It requires the key of the element to be deleted.
- If the specified key does not exist, it raises a KeyError.
- It can also be used to delete the entire dictionary.

2. Using pop() Method

- The pop() method removes the specified key-value pair from the dictionary.
- It returns the value associated with the deleted key.
- If the key does not exist, it raises a KeyError by default.
- We can provide a default value to avoid the error.

3. Using popitem() Method

- The popitem() method removes and returns the last inserted key-value pair from the dictionary.
- It returns the deleted key-value pair in the form of a tuple.
- If the dictionary is empty, it raises a KeyError.

4. Using clear() Method

- The clear() method removes all key-value pairs from the dictionary.
- It makes the dictionary empty.
- It does not delete the dictionary variable itself.
- It returns None.

8. Count the frequency of each word in a given sentence using a dictionary.

In [9]:
```
d = {1:"ram", 2: "ram", 3:"shaym"}
word = "ram"
l = list(d.values())
if word in l:
    print(l.count(word))
else:
    print(-1)
```

2

9. Count the frequency of each character in a given word using a dictionary.

In [17]:
```
d = {'r':"ram", 'a': "ram", 't':"shaym"}
word = "a"
s = "".join(list(d.keys()))
l = "".join(list(d.values()))

print(l.count(word) + s.count(word))
```

4

10. Sort a dictionary by keys and values.

In [8]:
```
d = {'r':"ram", 'a': "ram", 's':"shaym"}
dict(sorted(d.items()))
```

Out[8]: {'a': 'ram', 'r': 'ram', 's': 'shaym'}

11. Find the sum of all values in a dictionary

In [10]:
```
d = {1:1,2:2,3:3,4:4,5:5,6:6,7:7,8:8,9:9,0:0}
print(sum(d.values()))
```

```
45
```

12. Create a dictionary using dictionary comprehension that contains even numbers as keys and their squares as values from 1 to 10.

In [19]:
```
d = dict((x,x**2) for x in range(0,11,2))
print(d)
```

```
{0: 0, 2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

13. Group a list of words into lists of anagrams using a dictionary. Print the list of lists where each inner list contains words that are anagrams of each other.

● Input: ["listen", "silent", "enlist", "hello", "lemon", "melon", "debitcard", "badcredit"] ● Anagram groups: [['listen', 'silent', 'enlist'], ['hello'], ['lemon', 'melon'], ['debitcard', 'badcredit']]

In [36]:
```
l = ["listen", "silent", "enlist", "hello", "lemon", "melon", "debitcard", "badcred
dict = {}
for i in l:
    if dict.get("".join(sorted(i)),0) == 0:
        dict["".join(sorted(i))] = []
        a = dict["".join(sorted(i))]
        a.append(i)
    else:
        a = dict["".join(sorted(i))]
        a.append(i)

print(list(dict.values()))
```

```
[['listen', 'silent', 'enlist'], ['hello'], ['lemon', 'melon'], ['debitcard', 'badcr
edit']]
```

14. Merge two dictionaries. If there are common keys, sum their values. Print the merged dictionary.

In [42]:
```
d1 = {1:1,2:2,3:3,4:4,5:5}
d2 = {1:6,2:7,3:8,4:9,6:0}

s = set(list(d1.keys()))
s.update(list(d2.keys()))
```

```
for i in s:
    d1[i] = d1.get(i,0) + d2.get(i,0)

print(d1)
```

```
{1: 7, 2: 9, 3: 11, 4: 13, 5: 5, 6: 0}
```

15. Merge two lists into a dictionary using dictionary comprehension.

In [51]:
```
l1 = [1,2,3,4,5,6,2]
l2 = [4,5,6,7,8,9,0]
d = {i:j for i,j in zip(l1,l2)}
print(d)
```

```
{1: 4, 2: 0, 3: 6, 4: 7, 5: 8, 6: 9}
```

16. Find mirror characters in a string using a dictionary. For example, 'A' mirrors to 'Z', 'B' mirrors to 'Y', Input string: "Hello World"

In [70]:
```
l1 = ""
l2 = ""

for i in range(0,26):
    l1 += chr(ord('a') + i)
l2 = l1[::-1]
d = {k:v for k,v in zip(l1,l2)}
l1 = ""
l2 = ""

for i in range(0,26):
    l1 += chr(ord('A') + i)
l2 = l1[::-1]
d2 = {k:v for k,v in zip(l1,l2)}
d.update(d2)
string = "Hello World"
s = ""
for i in string:
    s += d.get(i,i)
print(s)
```

```
Svool Dliow
```

17. Convert a list of tuples into a dictionary.

In [73]:
```
l = [("a", 10), ("b", 20), ("c", 30)]

d = dict(l)

print(d)
```

```
{'a': 10, 'b': 20, 'c': 30}
```

18. Convert a dictionary to a list of tuples.

In [78]:
```
print(d)
l = list(d.items())
print(l)
```

```
{'a': 10, 'b': 20, 'c': 30}
[('a', 10), ('b', 20), ('c', 30)]
```

19. Create a dictionary that maps each lowercase and uppercase alphabet to its corresponding ASCII code.

In [81]:
```
l1 = ""
for i in range(0,26):
    l1 += chr(ord('a') + i)
    l1 += chr(ord('A') + i)
d = {i:ord(i) for i in l1}
print(d)
```

```
{'a': 97, 'A': 65, 'b': 98, 'B': 66, 'c': 99, 'C': 67, 'd': 100, 'D': 68, 'e': 101,
'E': 69, 'f': 102, 'F': 70, 'g': 103, 'G': 71, 'h': 104, 'H': 72, 'i': 105, 'I': 73,
'j': 106, 'J': 74, 'k': 107, 'K': 75, 'l': 108, 'L': 76, 'm': 109, 'M': 77, 'n': 11
0, 'N': 78, 'o': 111, 'O': 79, 'p': 112, 'P': 80, 'q': 113, 'Q': 81, 'r': 114, 'R':
82, 's': 115, 'S': 83, 't': 116, 'T': 84, 'u': 117, 'U': 85, 'v': 118, 'V': 86, 'w':
119, 'W': 87, 'x': 120, 'X': 88, 'y': 121, 'Y': 89, 'z': 122, 'Z': 90}
```

20. Count the number of vowels in a given string and store the counts in a dictionary.

In [82]:
```
d = {}
v = "aeiou"
string = "hello world hi i love icecream and pizza"
for i in string:
    if i in v:
        d[i] = string.count(i)
print(d)
```

```
{'e': 4, 'o': 3, 'i': 4, 'a': 3}
```

21. Create a nested dictionary where the keys are letters from 'A' to 'E' and the values are dictionaries mapping numbers from 1 to 5 to their squares.

Output = { 'A': {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}, 'B': {1: 1, 2: 4, 3: 9, 4: 16, 5: 25},...}

In [84]:
```
d = {}
for i in range(0,5):
    d[chr(ord('A') + i)] = dict((k,k*k) for k in range(1,6))
print(d)
```

```
{'A': {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}, 'B': {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}, 'C':
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}, 'D': {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}, 'E': {1: 1,
2: 4, 3: 9, 4: 16, 5: 25}}
```

22. What is a Set? List out all the properties of Set.

A Set in Python is a built-in data type used to store a collection of unique elements.

Sets are unordered collections, which means they do not maintain a fixed positional order of elements.

Sets are created using curly braces {} or the set() constructor.

Properties of Set in Python

1. Unordered: Sets do not maintain a guaranteed order of elements.
2. Unique Elements: Sets do not allow duplicate elements. Duplicate values are automatically removed.
3. Mutable: We can add or remove elements from a set after its creation.
4. Heterogeneous Elements: A set can contain elements of different data types, such as integers, strings, floats, and tuples.
5. Immutable Elements: Set elements must be immutable and hashable. Lists, dictionaries, and other sets cannot be stored directly inside a set.
6. No Indexing: Sets do not support indexing because they are unordered.
7. No Slicing: Sets do not support slicing.
8. Dynamic in Size: We can increase or decrease the size of a set at runtime.
9. No Nested Sets: A set cannot directly contain another mutable set. However, it can contain a frozenset (immutable set).
10. Empty Set: An empty set is created using set(), not {}, because {} creates an empty dictionary.
11. Mathematical Set Operations: Python supports union, intersection, difference, and symmetric difference operations on sets.
12. Membership Testing: We can check whether an element exists in a set using the in and not in operators.
13. Efficient Searching: Sets generally provide average O(1) time complexity for membership testing.

23. Explain the difference between a Set and a Frozenset in Python.

In Python, both set and frozenset are built-in data types used to store a collection of unique elements.

The main difference is that a set is mutable, whereas a frozenset is immutable.

1. Set

- A set is a mutable collection of unique elements.
- We can add, update, and remove elements after its creation.
- Set elements must be immutable and hashable.
- Sets do not support indexing and slicing.
- Sets are created using {} or the set() constructor.

2. Frozenset

- A frozenset is an immutable collection of unique elements.
- We cannot add, update, or remove elements after its creation.
- Its elements must be immutable and hashable.
- Frozensets do not support indexing and slicing.
- Frozensets are created using the frozenset() constructor.

24. Can you have a set of sets in Python? Explain with reasons.

No, we cannot directly create a set of sets in Python because a set is mutable and unhashable.

Reasons:

1. A set can contain only immutable and hashable elements.
2. Since a set is mutable, it is unhashable.
3. Therefore, we cannot store a normal set inside another set.
4. However, we can use frozenset instead because it is immutable and hashable.

25. How are sets different from lists and tuples in Python?

1. List

- A list is an ordered and mutable collection of elements.
- It allows duplicate elements.
- It supports indexing and slicing.
- Elements can be added, updated, and deleted.

2. Tuple

- A tuple is an ordered and immutable collection of elements.
- It allows duplicate elements.
- It supports indexing and slicing.
- Elements cannot be added, updated, or deleted after creation.

3. Set

- A set is an unordered collection of unique elements.

- It is mutable, but its elements must be immutable and hashable.
- It does not support indexing and slicing.
- It automatically removes duplicate elements.
- It supports mathematical operations such as union, intersection, difference, and symmetric difference.

26. Find the minimum and maximum elements in a set without using the built-in min() and max() functions.

In [87]:
```
s = {1,2,3,4,5,6,7,8,9}
m = None
mi = None
for i in s:
    if m is None or mi is None:
        m = mi = i
    elif m < i:
        m = i
    elif mi > i:
        mi = i
print(mi,m)
```

```
1 9
```

30. Perform set operations (union, intersection, difference) on sets containing both integers and strings without using built-in methods.

Set 1: {1, 2, 'apple', 'orange'} Set 2: {2, 3, 'orange', 'banana'}

In [89]:
```
s1 = {1, 2, 'apple', 'orange'}
s2 = {2, 3, 'orange', 'banana'}

print("union - ", s1.union(s2))
print("intersection - ", s1.intersection(s2))
print("difference - ", s1.difference(s2))
```

```
union -  {1, 2, 3, 'orange', 'banana', 'apple'}
intersection -  {2, 'orange'}
difference -  {1, 'apple'}
```

In [ ]:
```

```